

14.4 A 51.6TFLOPs/W Full-Datapath CIM Macro Approaching Sparsity Bound and $<2^{-30}$ Loss for Compound AI

Zhiheng Yue*, Xujiang Xiang*, Yang Wang, Ruiqi Guo, Huiming Han, Shaojun Wei, Yang Hu, Shouyi Yin

Tsinghua University, Beijing, China

*Equally Credited Authors (ECAs)

Large-language models (LLM) are widely used with exceptional performance, but their prohibitive size and cost limits deployment on edge devices. The compound-AI combines several specialized small models to achieve matched or even superior accuracy on target downstream tasks [1,2]: e.g., RevCol [3] fuses multiple convolutional models and surpasses the 7.2B-parameter monolithic LLM model [4] on the ImageNet classification task by 1.64%. Therefore, the shift to compound systems opens opportunities for edge deployment in addition to model-size scaling. SRAM-based floating point (FP) CIM promises accelerated edge-AI models [5-14]. Figure 14.4.1 shows a conventional FP CIM macro, which faces three challenges for compound-AI acceleration: (1) Previous FP CIMs rely on a Gaussian data distribution to reduce alignment loss, where data is distributed around its mean [11]. However, compound-AI employs diverse models, with multiple data distributions, leading to an accuracy degradation of more than 5% [15]. (2) In variable compound-AIs, the CIM only performs MAC operations [16-23]; however, partial-sum/residual accumulation requires data movement and accounts for 26-72% power consumption, especially for bit-serial CIMs. (3) Conventional CIMs explore sparsity according to specific model features [11,12], but compound-AI models have flexible sparse patterns, causing a gap between the actual and theoretical speedup (1/1-sparsity).

We address these challenges by implementing an FP-CIM macro with three key features: (1) A heterogeneous-CIM macro, which supports parallel mantissa multiplication and exponent accumulation, allows post-product alignment to approach $<2^{-30}$ accuracy loss. The proposed mantissa^{EE} format supports different data precision for lossless computing. (2) The full data path is computed in memory to minimize data-movement power, and a hybrid format is explored to achieve peak-energy efficiency. (3) A network-in-memory (NIM) and a dynamic launcher capture static and dynamic sparsity opportunities, achieving $>2.7\times$ performance improvement compared to prior work. Figure 14.4.2 presents the overall architecture of the proposed macro, which comprises of 16 heterogeneous-CIM cores to deploy specialized models. Each core includes 8 CIM-sets and an inter-set adder. A 0.5kb accumulate-in-memory is used for partial-sum and residual accumulation. The CIM-set performs 4 parallel FP16 or 8 FP8 MACs within a 1.375kb mantissa-exponent CIM (ME-CIM), also included is a 0.25kb NIM and dynamic launcher for sparsity-aware acceleration. This CIM macro is designed as an edge accelerator to accommodate representative compound-AI task models and improves the accuracy $\times$ power $\times$ performance FoM.

While conventional CIM macros adopt rail-to-rail logic for FP computing, the mantissa inputs have already been deteriorated, due to alignment shifting, especially when the data is widely distributed with diverse exponent values [5-6,8-9]. Figure 14.4.3 presents the proposed workflow, in which the mantissa and exponent are first computed in parallel in the CIM and the alignment is postponed when the CIM generates partial products. Therefore, the CIM-computing error is mitigated and only occur during CIM-product accumulation. A proof-of-concept illustrates that the maximum FP16-MAC error rate is lower than 2^{-30} , compared to 2^{-1} in an earlier FP macro [5][12]. This lossless CIM design adapts to different data precision with Mantissa^{EE} transforms. Specifically, the unpacked mantissa of FP16 is shifted by the least significant 2b of the exponent to form a 16b Mantissa^{EE}, which is compatible with FP8 and INT8 representations, and reduces the required exponent width for alignment. The Mantissa^{EE} and exponent are stored in the same row of the ME-CIM, and perform multiply/add with inputs in parallel. The array integrates the XOR/AND logic necessary for both a Booth multiplier and a full adder (FA). Based on the required data precision, the surrounding shifter/partial FA is activated to complete the remaining logic. Lossless CIM products are then accumulated after post-alignment, which shifts the mantissa according to the exponent difference. The Mantissa^{EE} format concentrates the exponent distribution, and the post-alignment further takes the benefits of a higher product width to reduce truncation loss. The signal-error ratio (SER) evaluates hardware accuracy: it is defined as $10\log_{10}(\text{signal}/\text{error})$ [24]. This macro demonstrates an average of 60.1dB SER across 64 $\times$ 64 samples, which is 3.14 $\times$ higher than a conventional FP CIM. Applied to compound models, ResNet-series, MobileNet-series, Attention-mechanism, and others, this work lowers the accuracy loss by 11.07 $\times$, on average.

Figure 14.4.4 shows the accumulate-in-memory (AIM) macro, which reduces data movement by performing in-situ accumulation. The data is stored in a 6T cell and uses $N_1/N_2/N_3$ for decoupled read and accumulation. The stored content is added with the CIM produced partial sum or residual connection within a pseudo-row. First, XOR logic distinguishes between INT and FP modes. Two elements are used for accumulation in INT mode. FP mode requires an extra subtraction to compare two operands for alignment. When

CWL and CIM_{EN} are activated, the stored data and input together decide the CBL/CBLB voltage; when the cell stores 1 and IN equals 1, only then the N_1 is cut-off and N_4 drives CBL to a high state after pre-charge; thus, realizing an AND operation. Symmetrically, CBLB performs a NOR operation. Next, a partial FA takes advantage of the produced logic to simplify the adder design, and the summation output is written back to the cell. In FP32 mode, the FA output is utilized for alignment, accumulation, and further rounding. Compared with a full-digital scheme, AIM saves 38.3% and 15.4% power in INT32 and FP32 modes. Deployed on variable models, AIM reduces system power by 22.1 and 9.7% in INT and FP modes.

The power consumption of the multiplication data path is optimized by leveraging the best combination of multiplier/multiplicand format. Prior energy efficient multipliers choose sign-magnitude (SM) in place of the 2's complement (2C) format, but these formats do not accommodate a CIM structure [25]. The Booth-multiplier in ME-CIM achieves bit-parallelism by encoding inputs and correspondingly toggling and shifting weights. Therefore, a hybrid 2C (IN) and SM (W) combination incurs the fewest bit toggles and avoids adding-1 compensation, which reduces toggle activity by 5.2-47.5%; thereby, reducing power by 6.3-42.7% compared to other formats.

Figure 14.4.5 shows the static-dynamic sparsity-aware accelerator used to adapt to flexible compound-model sparse patterns. The static router targets the stationary sparsity scenarios: e.g., weight pruning and attention masking. The most-widely accepted 2:4 compression is used to maintain compatibility across different models [26]. The NIM selects 2-input elements from each of the 4 cells, according to the matched weight. Two improbable route paths are noted and excluded to reduce network complexity. Readout data are then Booth encoded to perform weight multiplication. As Figure 14.4.5 illustrates, each term is represented using 4 fields: ID, sign, valid, and SF. The encoding terms contain random valid distributions: i.e. dynamic sparsity. A Booth pool-lane sharing (BPLS) mechanism relocates non-zero terms to approach the ideal speedup due to data sparsity: specifically, 8 or 4 parallel inputs each with 4 or 8 encoding terms form a Booth pool. The dynamic launcher collects valid Booth-pool fields and in parallel leading-1 detectors iterate to find valid terms in sequence. Valid terms are retained and applied to the CIM for selective Booth-encoding multiplication. The ID field selects matched weights from the array, XOR logic toggles the sign bit in the SM representation, AND logic masks any non-valid elements, and the SF field provides shifting offset. By combining static and dynamic sparsity techniques, this macro achieves a 5.30 $\times$ speedup on average on compound models: a 2 $\times$ static and 2.65 $\times$ dynamic speedup. Performance approaches 92.6% of the sparsity speedup bound. To further explore sparsity power benefits, a no pre-charge dual-port bit cell is implemented. The bit cell uses an inverter to eliminate the power-consuming pre-charge circuit. Charging and discharging power is consumed only when the RBL voltage changes. Therefore, this cell naturally suits sparse-data applications as RBLs are likely to read out consecutive 0s. Compared to an 8T dual-port cell, the 10T cell improves power consumption by 1.95 $\times$ with a 1.26 $\times$ area increase [27].

Figure 14.4.6 shows the test-chip measurement results. The test chip is fabricated in 28nm and occupies a 2.67mm². The 224kb heterogeneous macro integrates MAC, routing and accumulating operations for an all-in-memory data path; thus, achieving a peak efficiency of 16.2TFLOPs/W, and an average efficiency of 5.55 - 12.54TFLOPs/W when deployed on compound models. A post-product alignment enhances the signal-to-error ratio to 60.1dB, accomplishing an average error rate below 0.6% of all models in Figure 14.4.3. The NIM and launcher together explore static/dynamic sparsity to boost performance to nearly the speedup bound. A pre-charge-free cell also helps reduce access power for sparse data scenarios: allowing the macro to achieve 51.6TFLOPs/W and 2.44TFLOPs/mm². With all techniques, the proposed macro achieves a SER $\times$ system throughput/area / power FoM that is $>5.72\times$ higher than prior FP-CIMs. Figure 14.4.7 presents the die photograph.

Acknowledgement:

This work is supported in part by the National Key R&D Project under grant 2023YFB4403100; in part by the NSFC under grant 62125403, and grant 92164301; in part by the National Science and Technology Major Project under grant 2022ZD0115201; in part by the National Key Research and Development Program under grant 2021ZD0114400; Beijing S&T Project Z221100007722023; in part by the Beijing National Research Center for Information Science and Technology; and, in part by the Beijing Advanced Innovation Center for Integrated Circuits. (Corresponding author is Shouyi Yin, yinsy@tsinghua.edu.cn)

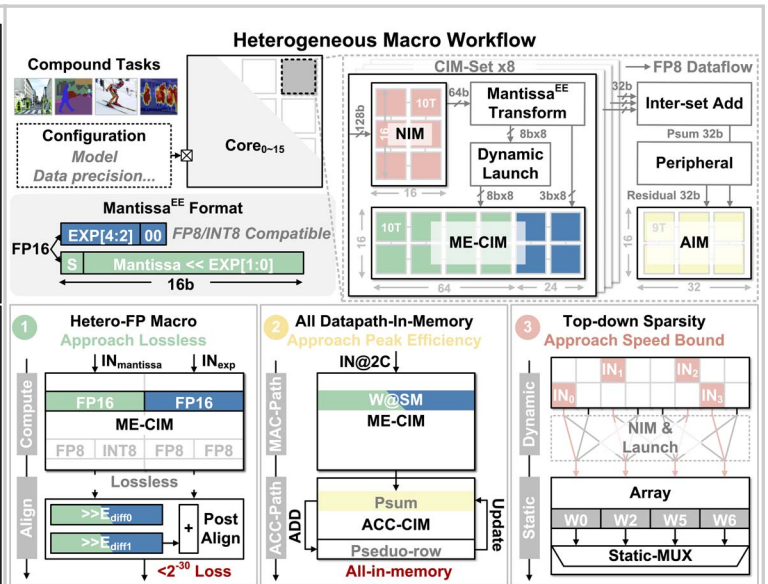


Figure 14.4.2: Overall architecture of heterogeneous macro and its three key features to approach peak accuracy, energy efficiency, speed.

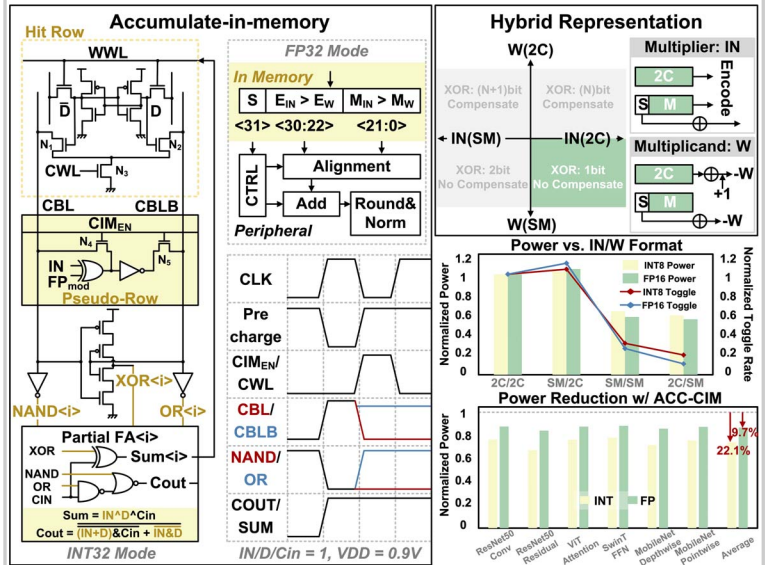


Figure 14.4.4: Accumulate-in-memory design and hybrid format multiplication optimization.

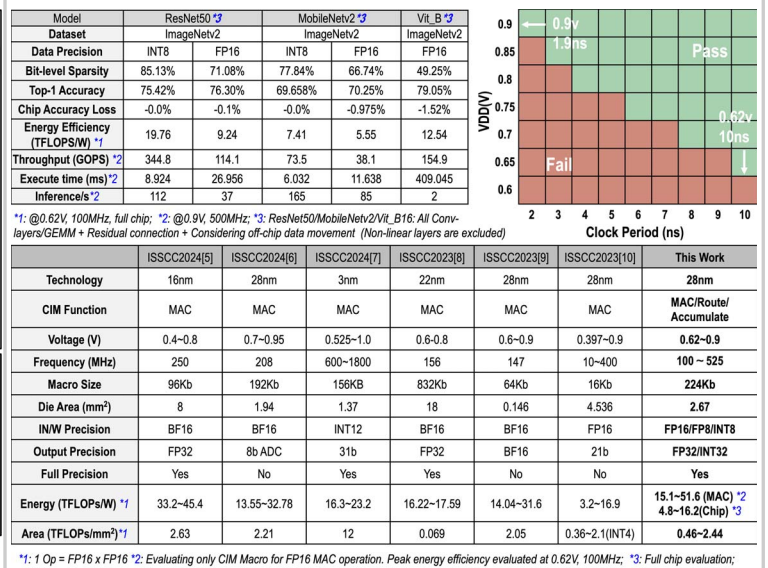


Figure 14.4.6: Measurement results for representative models; Shmoo Plot; Key-feature comparison table to prior work.

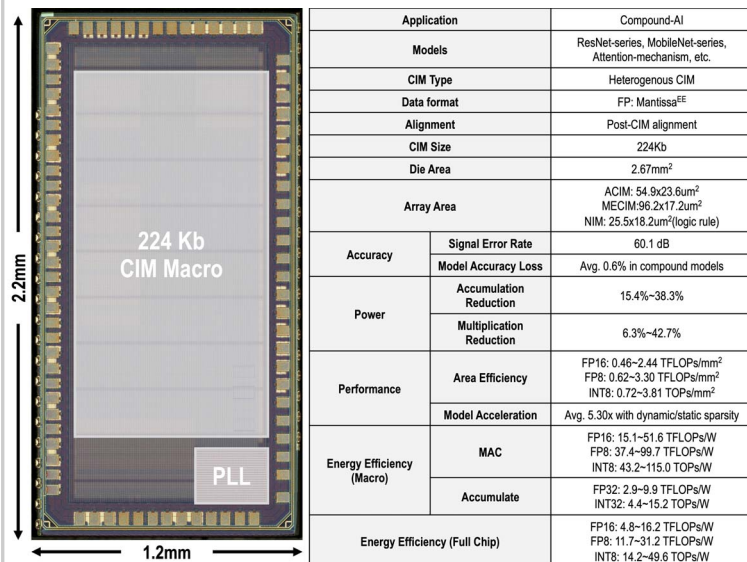


Figure 14.4.7: Die photo and hardware specification table.

References:

- [1] R. Prabhakar et al., "SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts," *arXiv: 2405.07518*, 2024.
- [2] M. Zaharia, et al., "The shift from models to compound ai systems," *URI: https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems*, 2024.
- [3] Y. Cai, et al., "Reversible Column Networks," *ICLR*, 2023.
- [4] C. Riquelme, et al., "Scaling Vision with Sparse Mixture of Experts," *NeurIPS 2021*, pp. 8583-8595.
- [5] W.-S. Khwa et al., "A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices," *ISSCC*, pp. 568-570, 2024.
- [6] Y. Yuan et al., "A 28nm 72.12TFLOPS/W Hybrid-Domain Outer-Product Based Floating-Point SRAM Computing-in-Memory Macro with Logarithm Bit-Width Residual ADC," *ISSCC*, pp. 576-578, 2024.
- [7] M.-E. Shih et al., "NVE: A 3nm 23.2TOPS/W 12b-Digital-CIM-Based Neural Engine for High-Resolution Visual-Quality Enhancement on Smart Devices," *ISSCC*, pp. 360-362, 2024.
- [8] P. -C. Wu et al., "A 22nm 832Kb Hybrid-Domain Floating-Point SRAM In-Memory-Compute Macro with 16.2-70.2TFLOPS/W for High-Accuracy AI-Edge Devices," *ISSCC*, pp. 126-128, 2023.
- [9] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W Digital-Domain Floating-Point-Computing-Unit and Double-Bit 6T-SRAM Computing-in-Memory Macro for Floating-Point CNNs," *ISSCC*, pp. 128-130, 2023.
- [10] J. Yue et al., "A 28nm 16.9-300TOPS/W Computing-in-Memory Processor Supporting Floating-Point NN Inference/Training with Intensive-CIM Sparse-Digital Architecture," *ISSCC*, pp. 252-254, 2023.
- [11] J. Yue et al., "STICKER-IM: A 65 nm Computing-in-Memory NN Processor Using Block-Wise Sparsity Optimization and Inter/Intra-Macro Data Reuse," *IEEE JSSC*, vol. 57, no. 8, pp. 2560-2573, Aug. 2022.
- [12] F. Tu et al., "ReDCIM: Reconfigurable Digital Computing- In -Memory Processor with Unified FP/INT Pipeline for Cloud AI Acceleration," *IEEE JSSC*, vol. 58, no. 1, pp. 243-255, Jan. 2023.
- [13] W. Tai-Hao, et al., "A 22nm 16Mb Floating-Point ReRAM Compute-in-Memory Macro with 31.2 TFLOPS/W for AI Edge Devices," *ISSCC*, pp. 580-582, 2024.
- [14] R. Guo et al., "A 28nm 74.34TFLOPS/W BF16 Heterogenous CIM-Based Accelerator Exploiting Denoising-Similarity for Diffusion Models," *ISSCC*, pp. 362-364, 2024.
- [15] C. Guo et al., "ANT: Exploiting Adaptive Numerical Data Type for Low-bit Deep Neural Network Quantization," *MICRO*, pp. 1414-1433, 2022.
- [16] G. Desoli et al., "A 40-310TOPS/W SRAM-Based All-Digital Up to 4b In-Memory Computing Multi-Tiled NN Accelerator in FD-SOI 18nm for Deep-Learning Edge Applications," *ISSCC*, pp. 260-262, 2023.
- [17] P. Chen et al., "A 22nm Delta-Sigma Computing-In-Memory ($\Delta\Sigma$ CIM) SRAM Macro with Near-Zero-Mean Outputs and LSB-First ADCs Achieving 21.38TOPS/W for 8b-MAC Edge AI Processing," *ISSCC*, pp. 140-142, 2023.
- [18] K. Ueyoshi et al., "DIANA: An End-to-End Energy-Efficient Digital and ANALog Hybrid Neural Network SoC," *ISSCC*, pp. 256-258, 2022.
- [19] H. Fujiwara et al., "A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² Fully-Digital Compute-in-Memory Macro Supporting INT12 $\times$ INT12 with a Parallel-MAC Architecture and Foundry 6T-SRAM Bit Cell," *ISSCC*, pp. 572-574, 2024.
- [20] Z. Jiang, S. Yin, J. -S. Seo and M. Seok, "C3SRAM: In-Memory-Computing SRAM Macro Based on Capacitive-Coupling Computing," in *IEEE SSCL*, vol. 2, no. 9, pp. 131-134, Sept. 2019.
- [21] H. Jia, H. Valavi, Y. Tang, J. Zhang and N. Verma, "A Programmable Heterogeneous Microprocessor Based on Bit-Scalable In-Memory Computing," in *IEEE JSSC*, vol. 55, no. 9, pp. 2609-2621, Sept. 2020.
- [22] S. Liu et al., "A 28nm 53.8TOPS/W 8b Sparse Transformer Accelerator with In-Memory Butterfly Zero Skipper for Unstructured-Pruned NN and CIM-Based Local-Attention-Reusable Engine," *ISSCC*, pp. 250-252, 2023.
- [23] H. Zhu et al., "COMB-MCM: Computing-on-Memory-Boundary NN Processor with Bipolar Bitwise Sparsity Optimization for Scalable Multi-Chiplet-Module Edge Machine Learning," *ISSCC*, pp. 250-252, 2022.
- [24] S. Kim et al., "Scaling-CIM: An eDRAM-based In-Memory-Computing Accelerator with Dynamic-Scaling ADC for SQNR-Boosting and Layer-wise Adaptive Bit-Truncation," *IEEE Symp. VLSI Circuits*, 2023..
- [25] H. An et al., "An 8.09TOPS/W Neural Engine Leveraging Bit-Sparsified Sign-Magnitude Multiplications and Dual Adder Trees," *ISSCC*, pp. 422-424, 2023.
- [26] J. Choquette, E. Lee, R. Krashinsky, V. Balan and B. Khailany, "The A100 Datacenter GPU and Ampere Architecture," *ISSCC*, pp. 48-50, 2021.
- [27] Hiroki Noguchi et al., "Which is the best dual-port SRAM in 45-nm process technology? — 8T, 10T single end, and 10T differential —" *ICICDT*, pp. 55-58, 2008.